

Experiment 'p_2_8_rectangle_properties' Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 5

Hard fails (resolution): 1

Soft fails (resolution): 1

Verification attempts: 3

Problem Specification

Problem name: p_2_8_rectangle_properties

Natural language statement: Write a method that takes the lengths of the sides of a rectangle. Then return the area and perimeter of the rectangle and the length of the diagonal (use the Pythagorean theorem) with a residual error less than 0.01.

Method signature: p_2_8_rectangle_properties(length: real, width: real) returns (area: real, perimeter: real, diagonal: real)

Ensures

- `area == length * width`
- `perimeter == 2.0 * (length + width)`
- `-0.01 < (diagonal * diagonal) - ((length * length) + (width * width)) < 0.01`

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes the lengths of the sides of a rectangle. Then return the area
→ and perimeter of the rectangle and the length of the diagonal (use the Pythagorean
→ theorem) with a residual error less than 0.01.

The signature should be:

p_2_8_rectangle_properties(length: real, width: real) returns (area: real, perimeter:
→ real, diagonal: real)

The method should respect the following contract:

ensures area == length * width, ensures perimeter == 2.0 * (length + width), ensures
→ $-0.01 < (\text{diagonal} * \text{diagonal}) - ((\text{length} * \text{length}) + (\text{width} * \text{width})) < 0.01$

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdY7yGrU6LXn0OV8GZZlnC3Mbbbx

Model: gpt-5.5-2026-04-23

Created at: 1780484143

Finish reason: stop

Usage: 434 tokens in, and 1162 tokens out

```
{
  area := length * width;
  perimeter := 2.0 * (length + width);

  var s := length * length + width * width;
  assert 0.0 <= length * length;
  assert 0.0 <= width * width;
  assert 0.0 <= s;
  assert exists d: real :: d * d == s;
  diagonal :| diagonal * diagonal == s;
  assert (diagonal * diagonal) - s == 0.0;
}
```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```
problem_data/p_2_8_rectangle_properties/program.dfy(13,9): Warning: Could not find a
  ↳ trigger for this quantifier. Without a trigger, the quantifier may cause brittle
  ↳ verification. To silence this warning, add an explicit trigger using the {:trigger
  ↳ } attribute. For more information, see the section on quantifier instantiation
  ↳ rules in the reference manual.
```

```
|
13 | assert exists d: real :: d * d == s;
| ~~~~~
```

Compilation failed because warnings were found and --allow-warnings is false

Correct the error in code by altering only the method body. Produce and show only the

- ↳ Dafny body, including the curly braces that surround it. Do not show the signature
- ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdYaWXIAScgiigIGMQgYxXyecUg5

Model: gpt-5.5-2026-04-23

Created at: 1780484172

Finish reason: stop

Usage: 730 tokens in, and 3069 tokens out

```
{
  area := length * width;
  perimeter := 2.0 * (length + width);

  var s := length * length + width * width;
  assert 0.0 <= length * length;
  assert 0.0 <= width * width;
  assert 0.0 <= s;
  diagonal :| diagonal * diagonal == s;
  assert (diagonal * diagonal) - s == 0.0;
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_2_8_rectangle_properties/program.dfy(13,11): Error: cannot establish the
  ↳ existence of LHS values that satisfy the such-that predicate. Note, no trigger was
  ↳ found for the such-that predicate, which may be the reason the proof failed. To
  ↳ give a trigger explicitly, use the {:trigger} attribute. For more information, see
  ↳ the section on quantifier instantiation rules in the reference manual.
|
13 | diagonal :| diagonal * diagonal == s;
| ^^
```

Dafny program verifier finished with 0 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmdZjWgYiKUif4jtzuy5SssTEQlrD

Model: gpt-5.5-2026-04-23

Created at: 1780484243

Finish reason: stop

Usage: 1046 tokens in, and 5312 tokens out

```
{
  area := length * width;
  perimeter := 2.0 * (length + width);

  var s := length * length + width * width;
  assert 0.0 <= length * length;
  assert 0.0 <= width * width;
  assert 0.0 <= s;
  assert exists d: real {:trigger d * d} :: d * d == s;
  diagonal :| diagonal * diagonal == s;
  assert (diagonal * diagonal) - s == 0.0;
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_2_8_rectangle_properties/program.dfy(13,2): Error: assertion might not
  → hold
|
13 | assert exists d: real {:trigger d * d} :: d * d == s;
    | ~~~~~
```

Dafny program verifier finished with 0 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dmdbk8uEROJ2HPdelHnH2nfajnp5

Model: gpt-5.5-2026-04-23

Created at: 1780484368

Finish reason: stop

Usage: 1272 tokens in, and 4269 tokens out

```
{
  area := length * width;
  perimeter := 2.0 * (length + width);

  var s := length * length + width * width;
  assert 0.0 <= length * length;
  assert 0.0 <= width * width;
  assert 0.0 <= s;

  diagonal := Std.Math.Sqrt(s);
  assert diagonal * diagonal == s;
  assert -0.01 < (diagonal * diagonal) - s < 0.01;
}
```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_2_8_rectangle_properties/program.dfy(14,14): Error: unresolved identifier:

```
    ↪ Std
    |
14 | diagonal := Std.Math.Sqrt(s);
    | ~~~
```

1 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

- ↪ Dafny body, including the curly braces that surround it. Do not show the signature
- ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmddS0mYKam1Tr9pXBKZQg1IoVdKW

Model: gpt-5.5-2026-04-23

Created at: 1780484474

Finish reason: stop

Usage: 1506 tokens in, and 11054 tokens out

```
{
  area := length * width;
  perimeter := 2.0 * (length + width);

  var s := length * length + width * width;
  assert 0.0 <= length * length;
  assert 0.0 <= width * width;
  assert 0.0 <= s;
```

```

var hi := 1.0;
while hi * hi <= s
  invariant 1.0 <= hi
  invariant 0.0 <= s
  decreases s - hi
{
  assert hi <= hi * hi;
  assert hi <= s;
  hi := 2.0 * hi;
}

var lo := 0.0;
var H := hi;
var factor := 1.0;
var target := 256.0 * H * H;

while factor < target
  invariant 1.0 <= H
  invariant 1.0 <= factor
  invariant 0.0 <= lo <= hi <= H
  invariant lo * lo <= s < hi * hi
  invariant (hi - lo) * factor <= H
  invariant target == 256.0 * H * H
  decreases target - factor
{
  var oldLo := lo;
  var oldHi := hi;
  var oldFactor := factor;
  var mid := (oldLo + oldHi) / 2.0;

  assert oldLo <= mid <= oldHi;
  if mid * mid <= s {
    lo := mid;
  } else {
    hi := mid;
  }

  assert hi - lo == (oldHi - oldLo) / 2.0;
  factor := 2.0 * oldFactor;
  assert (hi - lo) * factor == (oldHi - oldLo) * oldFactor;
}

assert target <= factor;
assert 0.0 <= hi - lo;
assert (hi - lo) * target <= (hi - lo) * factor;
assert (hi - lo) * target <= H;
assert 0.0 < H;
assert target == 256.0 * H * H;
assert ((hi - lo) * 256.0 * H) * H == (hi - lo) * target;
assert (hi - lo) * 256.0 * H <= 1.0;

```

```

assert hi + lo <= 2.0 * H;
assert (hi - lo) * (hi + lo) <= (hi - lo) * (2.0 * H);
assert (hi - lo) * (hi + lo) <= 1.0 / 128.0;
assert 1.0 / 128.0 < 0.01;
assert hi * hi - lo * lo == (hi - lo) * (hi + lo);
assert hi * hi - lo * lo < 0.01;

diagonal := lo;
assert diagonal * diagonal <= s;
assert s < hi * hi;
assert s - diagonal * diagonal < 0.01;
assert -0.01 < diagonal * diagonal - s;
assert diagonal * diagonal - s < 0.01;
}

```

Final Program

```

method p_2_8_rectangle_properties(length: real, width: real) returns (area: real,
    ↪ perimeter: real, diagonal: real)
    ensures area == length * width
    ensures perimeter == 2.0 * (length + width)
    ensures -0.01 < (diagonal * diagonal) - ((length * length) + (width * width)) <
        ↪ 0.01
{
    area := length * width;
    perimeter := 2.0 * (length + width);

    var s := length * length + width * width;
    assert 0.0 <= length * length;
    assert 0.0 <= width * width;
    assert 0.0 <= s;

    var hi := 1.0;
    while hi * hi <= s
        invariant 1.0 <= hi
        invariant 0.0 <= s
        decreases s - hi
    {
        assert hi <= hi * hi;
        assert hi <= s;
        hi := 2.0 * hi;
    }

    var lo := 0.0;
    var H := hi;
    var factor := 1.0;
    var target := 256.0 * H * H;

    while factor < target
        invariant 1.0 <= H

```

```

invariant 1.0 <= factor
invariant 0.0 <= lo <= hi <= H
invariant lo * lo <= s < hi * hi
invariant (hi - lo) * factor <= H
invariant target == 256.0 * H * H
decreases target - factor
{
  var oldLo := lo;
  var oldHi := hi;
  var oldFactor := factor;
  var mid := (oldLo + oldHi) / 2.0;

  assert oldLo <= mid <= oldHi;
  if mid * mid <= s {
    lo := mid;
  } else {
    hi := mid;
  }

  assert hi - lo == (oldHi - oldLo) / 2.0;
  factor := 2.0 * oldFactor;
  assert (hi - lo) * factor == (oldHi - oldLo) * oldFactor;
}

assert target <= factor;
assert 0.0 <= hi - lo;
assert (hi - lo) * target <= (hi - lo) * factor;
assert (hi - lo) * target <= H;
assert 0.0 < H;
assert target == 256.0 * H * H;
assert ((hi - lo) * 256.0 * H) * H == (hi - lo) * target;
assert (hi - lo) * 256.0 * H <= 1.0;
assert hi + lo <= 2.0 * H;
assert (hi - lo) * (hi + lo) <= (hi - lo) * (2.0 * H);
assert (hi - lo) * (hi + lo) <= 1.0 / 128.0;
assert 1.0 / 128.0 < 0.01;
assert hi * hi - lo * lo == (hi - lo) * (hi + lo);
assert hi * hi - lo * lo < 0.01;

diagonal := lo;
assert diagonal * diagonal <= s;
assert s < hi * hi;
assert s - diagonal * diagonal < 0.01;
assert -0.01 < diagonal * diagonal - s;
assert diagonal * diagonal - s < 0.01;
}

```


Total Token Usage

Input tokens: 4988

Output tokens: 24866

Reasoning tokens: 23352

Sum of ‘total tokens’: 29854

Experiment Timings

Overall Experiment started at 1780484142817, ended at 1780484748023, lasting 605206ms (605.21 seconds)

Iteration #4 started at 1780484367476, ended at 1780484473664, lasting 106188ms (106.19 seconds)

Iteration #5 started at 1780484473664, ended at 1780484748023, lasting 274359ms (274.36 seconds)

Iteration #1 started at 1780484142817, ended at 1780484171937, lasting 29120ms (29.12 seconds)

Iteration #2 started at 1780484171938, ended at 1780484243481, lasting 71543ms (71.54 seconds)

Iteration #3 started at 1780484243482, ended at 1780484367476, lasting 123994ms (123.99 seconds)